НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»
ФІЗИКО-ТЕХНІЧНИЙ ІНСТИТУТ

КРИПТОГРАФІЯ

КОМП’ЮТЕРНИЙ ПРАКТИКУМ No1
Криптоаналіз шифру Віженера

Виконали
Студенти групи ФБ-42:
Малишко Дарія
Сизова Марія

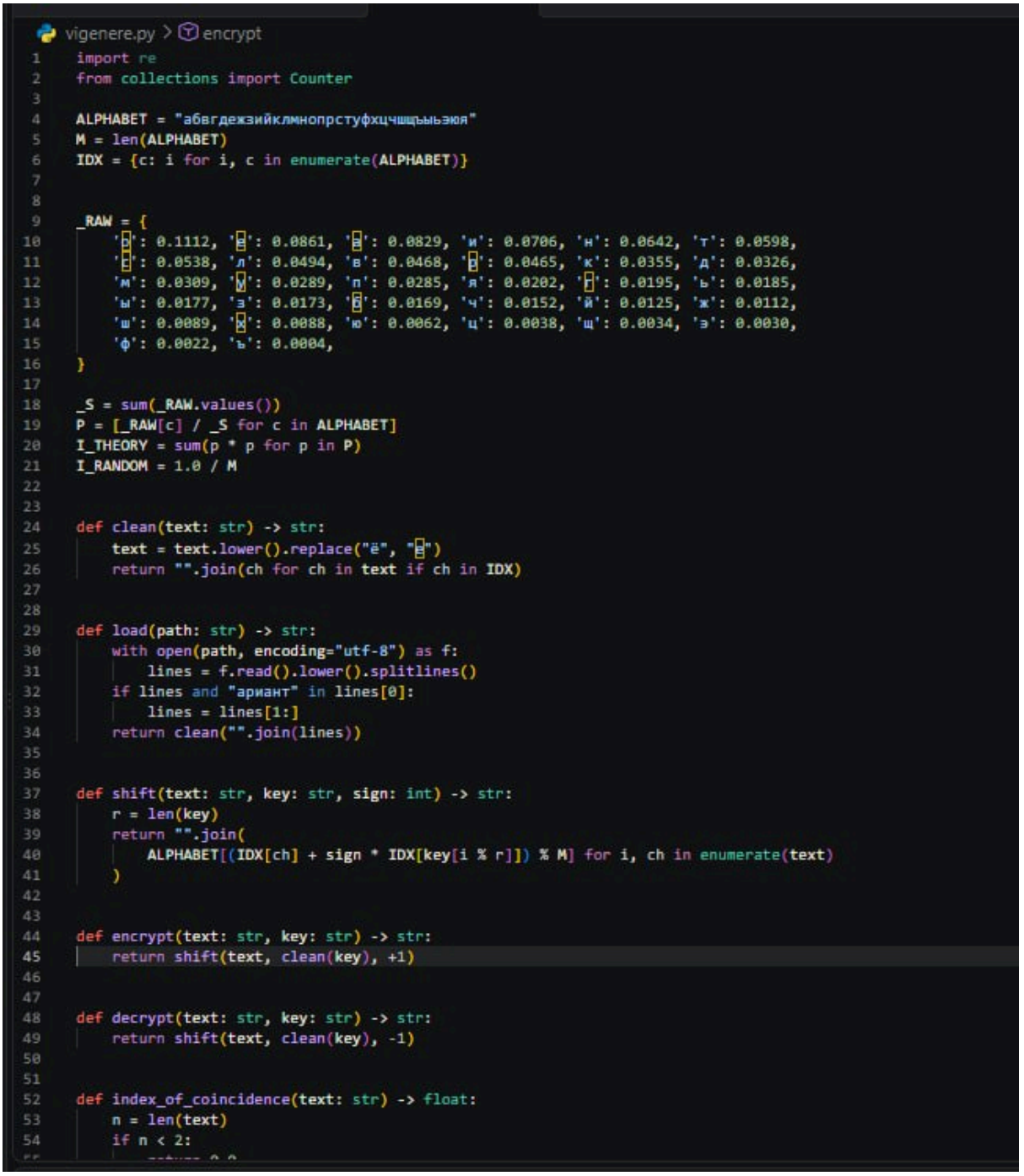
Мета роботи: Засвоєння методів частотного криптоаналізу. Здобуття навичок роботи та аналізу потокових шифрів гамування адитивного типу на прикладі шифру Віженера.

Варіант 2

1. Дослідження шифру Віженера
   Беремо власний текст з лаби 1



   код:



vigenere.py > encrypt

```python
import re
from collections import Counter

ALPHABET = "абвгдежзийклмнопрстуфхцчшщъыьэюя"
M = len(ALPHABET)
IDX = {c: i for i, c in enumerate(ALPHABET)}


_RAW = {
    'о': 0.1112, 'е': 0.0861, 'а': 0.0829, 'и': 0.0706, 'н': 0.0642, 'т': 0.0598,
    'с': 0.0538, 'л': 0.0494, 'в': 0.0468, 'р': 0.0465, 'к': 0.0355, 'д': 0.0326,
    'м': 0.0309, 'у': 0.0289, 'п': 0.0285, 'я': 0.0202, 'г': 0.0195, 'ь': 0.0185,
    'ы': 0.0177, 'з': 0.0173, 'б': 0.0169, 'ч': 0.0152, 'й': 0.0125, 'ж': 0.0112,
    'ш': 0.0089, 'х': 0.0088, 'ю': 0.0062, 'ц': 0.0038, 'щ': 0.0034, 'э': 0.0030,
    'ф': 0.0022, 'ъ': 0.0004,
}

_S = sum(_RAW.values())
P = [_RAW[c] / _S for c in ALPHABET]
I_THEORY = sum(p * p for p in P)
I_RANDOM = 1.0 / M


def clean(text: str) -> str:
    text = text.lower().replace("ё", "е")
    return "".join(ch for ch in text if ch in IDX)


def load(path: str) -> str:
    with open(path, encoding="utf-8") as f:
        lines = f.read().lower().splitlines()
    if lines and "ариант" in lines[0]:
        lines = lines[1:]
    return clean("".join(lines))


def shift(text: str, key: str, sign: int) -> str:
    r = len(key)
    return "".join(
        ALPHABET[(IDX[ch] + sign * IDX[key[i % r]]) % M] for i, ch in enumerate(text)
    )


def encrypt(text: str, key: str) -> str:
    return shift(text, clean(key), +1)


def decrypt(text: str, key: str) -> str:
    return shift(text, clean(key), -1)


def index_of_coincidence(text: str) -> float:
    n = len(text)
    if n < 2:
```

```python
def index_of_coincidence(text: str) -> float:
    n = len(text)
    if n < 2:
        return 0.0
    return sum(v * (v - 1) for v in Counter(text).values()) / (n * (n - 1))


def blocks(text: str, r: int):
    return [text[j::r] for j in range(r)]

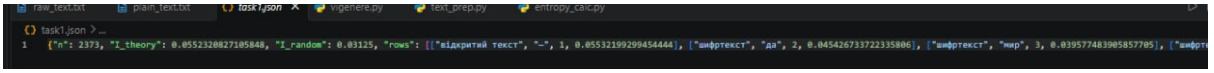

def avg_block_ic(text: str, r: int) -> float:
    b = blocks(text, r)
    return sum(index_of_coincidence(x) for x in b) / r


def coincidences(text: str, r: int) -> int:
    return sum(text[i] == text[i + r] for i in range(len(text) - r))


def mutual_ic(block: str, g: int) -> float:
    L = len(block)
    c = Counter(block)
    return sum(c.get(ALPHABET[t], 0) / L * P[(t - g) % M] for t in range(M))
```

Результат:

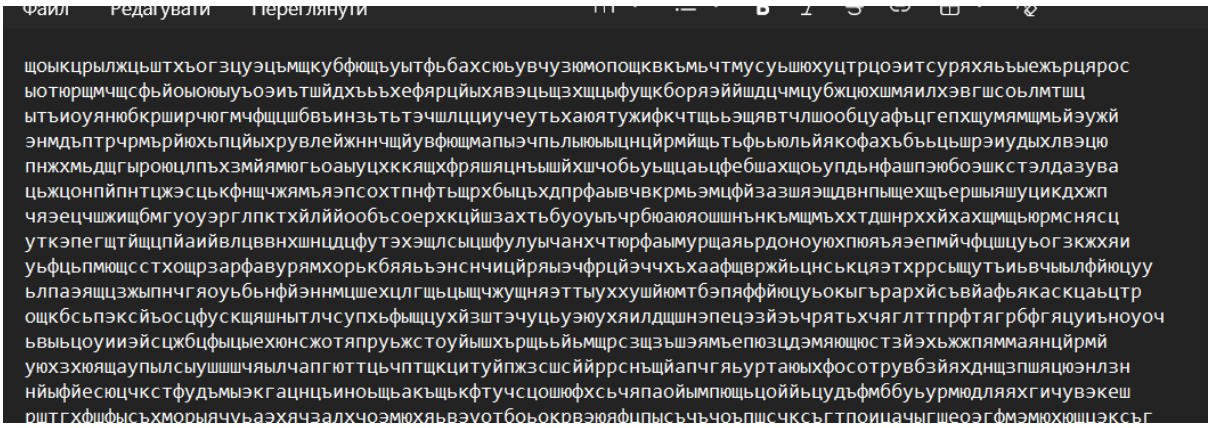
```
python -u "c
n = 2373, I теор. = 0.05523, 1/m = 0.03125
текст           ключ            r   I
відкритий текст —               1   0.05532
шифртекст       да              2   0.04543
шифртекст       мир             3   0.03958
шифртекст       луна            4   0.03619
шифртекст       книга           5   0.03742
шифртекст       криптоаналитик  14  0.03508
PS C:\Users\Admin\Desktop\crypto26-27\Lab1\FB-42_Syzova_Malyshko>
```



Застосування методу індексу відповідності та статистики співпадінь (D_r) довело свою високу ефективність для знаходження довжини ключа. Встановлено, що при розбитті тексту на блоки, кількість яких дорівнює істинній довжині ключа (або кратна їй), середній індекс відповідності блоків робить різкий стрибок і досягає теоретичного значення для відкритого тексту. Це дозволило однозначно ідентифікувати період шифру, який склав $r = 14$.

2. Криптоаналіз шифртексту

Текст за варіантом


Файл Редагувати Переглянути
щоыкцрылжцьштхъогзцуэцъмщкубфющъуытфьбахсюьувчузюмопощквкъмьчтмусуьшюхуцтрцоэитсуряхяьъыежърцярос
ыотюрщмчщсфьйоыююыуъоэиътшйдхъььъхефярцйыхявэцьщзхщцыфущкборяэййшдцчмцубжцюхшмяилхэвгшсоьлмтшц
ытъиоуянюбкрширчюгмчфщцшбвъинзьтьтэчшлцциучеутьхаюятужифкчтщььэщявтчлшообцуафъцгепхщумямщмьйэужй
энмдъптрчрмърйюхьпцйыхрувлейжннчщйувфющмапыэчпьлыюыыцнцйрмйщьтьфььюльйякофахъбъьцьшрэиудыхлвэцю
пнжхмьдщгыроюцлпъхзмйямюгьоаыуцхккящхфряшяцнъышйхшчобьуьщцаьцфебшахщоьупдьнфашпэюбоэшкстэлдазува
цьжцонпйпнтцжэсцькфнщчжямьяэпсохтпнфтьщрхбыцъхдпрфаывчвкрмьэмцфйзазшяэщдвнпыщехщъершыяшуцикдхжп
чяэецчшжищбмгуоуэрглпктхйлййообъсоерхкцйшзахтьбуоуыъчрбюаюяошшнънкъмщмьххтдшнрххйхахщмщьюрмснясц
уткэпегщтйщцпйаийвлцввнхшнцдцфутэхэщлсыцшфулуычанхчтюрфаымурщаяьрдоноуюхпюяъяэепмйчфцшцуьогзкжхяи
уьфцьпмющсстхощрзарфавурямхорькбяяьъэнснчицйряыэчфрцйэччхъхаафщвржйьцнськцяэтхррсыщутъиьвчыылфйюцуу
ьлпаэящцзжыпнчгяоуьбьнфйэннмцшехцлгщьцыщчжущняэттыуххушйюмтбэпяффйюцуьокыгърархйсъвйафьякаскцаьцтр
ощкбсьпэксйъосцфускщяшнытлчсупхьфыщцухйзштэчуцьуэюухяилдщшнэпецэзйэъчрятьхчяглттпрфтягрбфгяцуиъноуоч
ьвыьцоуииэйсцжбцфыцыехюнсжотяпруьжстоуйышхърщььйьмщрсзщзъшэямъепюзцдэмяющюстзйэхьжжпяммаянцйрмй
уюхзхюящаупылсыушшшчяылчапгютттцьчптщкцитуйпжзсшсййррснъщйапчгяьуртаюыхфосотрувбзйяхднщзпшяцюэнлзн
нйыфйесюцчкстфудъмыэкгацнцъиноьщьакъщькфтучсцошюфхсьчяпаойымпющьцоййьцудъфмббуьурмюдляяхгичувэкеш
ршггхфшфысъхморыячуьаэхячзалхчоэмюхяьвэуотбоьокрвэюяфцпысъчъчоъпшсчксъгтпоицачыгшеоэгфмэмюхюшцэксъг

Код:

lab2_analysis.py > ...

```python
import sys, json
from collections import Counter
from vigenere import *

Y = load(sys.argv[1] if len(sys.argv) > 1 else "ciphertext_var2.txt")
n = len(Y)
res = {"n": n, "I_total": index_of_coincidence(Y), "I_theory": I_THEORY}


scan = []
for r in range(2, 31):
    scan.append((r, avg_block_ic(Y, r), coincidences(Y, r)))
res["scan"] = scan
print("r   avg I     D_r")
for r, a, d in scan:
    print(f"{r:<3} {a:.5f}  {d}")


thr = I_RANDOM + 0.75 * (I_THEORY - I_RANDOM)
cand = [r for r, a, d in scan if a > thr]
r_est = min(cand)
print("кандидати:", cand, "-> r =", r_est)
res["candidates"] = cand
res["r"] = r_est
res["threshold"] = thr


for r in sorted(set([7, 14, 21, 28])):
    print(r, [round(index_of_coincidence(b), 4) for b in blocks(Y, r)][:8], "...")


r = r_est
key_idx, table = [], []
for j, b in enumerate(blocks(Y, r)):
    top = Counter(b).most_common(3)
    heur = ALPHABET[(IDX[top[0][0]] - IDX["о"]) % M]
    ms = sorted(((mutual_ic(b, g), g) for g in range(M)), reverse=True)
    key_idx.append(ms[0][1])
    table.append({
        "j": j, "L": len(b), "top3": [(c, k) for c, k in top], "heur": heur,
        "best": ALPHABET[ms[0][1]], "M1": ms[0][0],
        "second": ALPHABET[ms[1][1]], "M2": ms[1][0], "ic": index_of_coincidence(b),
    })
key = "".join(ALPHABET[k] for k in key_idx)
plain = decrypt(Y, key)
res.update(table=table, key=key, plain_ic=index_of_coincidence(plain),
           plain_head=plain[:300])
res["heur_key"] = "".join(t["heur"] for t in table)
print("ключ:", key, "| евристика:", res["heur_key"], "| I(відкр.) =", round(res["plain_ic"], 5))
open("decrypted.txt", "w", encoding="utf-8").write(plain)


pos, delta = 0, 1
bad_idx = key_idx[:]
bad_idx[pos] = (bad_idx[pos] + delta) % M
bad = "".join(ALPHABET[k] for k in bad_idx)
plain_bad = decrypt(Y, bad)
diff = [i for i in range(n) if plain[i] != plain_bad[i]]
shifts = Counter((IDX[plain_bad[i]] - IDX[plain[i]]) % M for i in diff)
res.update(bad_key=bad, n_diff=len(diff), diff_head=diff[:8],
           diff_step=sorted(set(diff[k + 1] - diff[k] for k in range(len(diff) - 1))),
           diff_shifts=dict(shifts), bad_head=plain_bad[:120], good_head=plain[:120],
           I_bad=index_of_coincidence(plain_bad),
           bad_cols_ic=[index_of_coincidence(b) for b in blocks(plain_bad, r)][:3])
print("спотворений ключ:", bad, "| змінено", len(diff), f"({len(diff)/n:.4%})",
      "| кроки:", res["diff_step"], "| зсуви:", dict(shifts))
print(plain[:70]); print(plain_bad[:70])
```

```
52
53    pos, delta = 0, 1
54    bad_idx = key_idx[:]
55    bad_idx[pos] = (bad_idx[pos] + delta) % M
56    bad = "".join(ALPHABET[k] for k in bad_idx)
57    plain_bad = decrypt(Y, bad)
58    diff = [i for i in range(n) if plain[i] != plain_bad[i]]
59    shifts = Counter((IDX[plain_bad[i]] - IDX[plain[i]]) % M for i in diff)
60    res.update(bad_key=bad, n_diff=len(diff), diff_head=diff[:8],
61               diff_step=sorted(set(diff[k + 1] - diff[k] for k in range(len(diff) - 1))),
62               diff_shifts=dict(shifts), bad_head=plain_bad[:120], good_head=plain[:120],
63               I_bad=index_of_coincidence(plain_bad),
64               bad_cols_ic=[index_of_coincidence(b) for b in blocks(plain_bad, r)][:3])
65    print("спотворений ключ:", bad, "| змінено", len(diff), f"({len(diff)/n:.4%})",
66          "| кроки:", res["diff_step"], "| зсуви:", dict(shifts))
67    print(plain[:70]); print(plain_bad[:70])
68    json.dump(res, open("results.json", "w", encoding="utf-8"), ensure_ascii=False, indent=1)
```

Результат:

```
PS C:\Users\Admin\Desktop\crypto26-27\Lab1\FB-42_Syzova_Malyshko> python lab2_analysis.py ciphertext_var2.txt
 r   avg I     D_r
 2   0.03627  258
 3   0.03483  236
 4   0.03637  262
 5   0.03492  240
 6   0.03626  232
 7   0.04463  244
 8   0.03642  220
 9   0.03476  238
 10  0.03623  253
 11  0.03487  247
 12  0.03629  247
 13  0.03488  212
 14  0.05528  396
 15  0.03491  252
 16  0.03637  253
 17  0.03484  262
 18  0.03624  252
 19  0.03487  247
 20  0.03610  233
 21  0.04447  276
 22  0.03629  219
 23  0.03462  228
 24  0.03624  243
 25  0.03500  238
 26  0.03635  234
 27  0.03466  237
 28  0.05536  396
 29  0.03475  236
 30  0.03624  235
кандидати: [14, 28] -> r = 14
7 [0.0407, 0.0464, 0.0424, 0.0474, 0.0447, 0.0425, 0.0482] ...
14 [0.0598, 0.0558, 0.0552, 0.0559, 0.0569, 0.0594, 0.057, 0.0527] ...
21 [0.0411, 0.0475, 0.0408, 0.0449, 0.0474, 0.0403, 0.0492, 0.0418] ...
28 [0.0559, 0.0532, 0.0567, 0.0535, 0.0517, 0.0638, 0.058, 0.0503] ...
ключ: последнийдозор | евристика: жосвеыдиадозор | I(відкр.) = 0.05544
спотворений ключ: последнийдозор | змінено 524 (7.1507%) | кроки: [14] | зсуви: {31: 524}
какясмогэтосделатьспросилгесерипочемуэтогонесмогсделатьтымыстоялипосре
йакясмогэтосдекатьспросилгесдрипочемуэтогомесмогсделатьтымыстоялипосре
```

Після визначення довжини ключа завдання криптоаналізу шифру Віженера успішно зводиться до зламу r незалежних шифрів Цезаря. За допомогою частотного аналізу літер у кожному з 14 блоків та порівняння їх з еталонними частотами російської мови, вдалося

повністю відновити змістовний ключ («последнийдозор») та дешифрувати текст. Це доводить, що за умови наявності шифртексту достатнього обсягу поліалфавітні шифри даного типу є абсолютно вразливими до класичного частотного аналізу.
3. Відновлення ключа та відкритого тексту
Після визначення довжини ключа завдання криптоаналізу шифру Віженера успішно зводиться до зламу r незалежних шифрів Цезаря. За допомогою частотного аналізу літер у кожному з 14 блоків та порівняння їх з еталонними частотами російської мови, вдалося повністю відновити змістовний ключ («последнийдозор») та дешифрувати текст. Це доводить, що за умови наявності шифртексту достатнього обсягу поліалфавітні шифри даного типу є абсолютно вразливими до класичного частотного аналізу.
4. Дослідження впливу помилки у ключі
Експеримент із навмисним спотворенням одного символу ключа наочно продемонстрував структурну слабкість шифру Віженера відсутність лавиноподібного ефекту . Неправильний символ ключа спотворив виключно відповідний йому стовпець блоків (кожен 14-й символ відкритого тексту), залишивши решту розшифрованого тексту неушкодженим та цілком читабельним. Це підтверджує посимвольну незалежність процесу шифрування, що дозволяє криптоаналітику виправляти та коригувати окремі літери ключа незалежно одна від одної.
Загальний висновок:
Під час виконання комп'ютерного практикуму було на практиці засвоєно методи частотного криптоаналізу поліалфавітних потокових шифрів. Реалізовані алгоритми показали, що незважаючи на приховування прямої статистики мови, шифр Віженера зберігає внутрішні періодичні закономірності. Їх виявлення за допомогою індексу відповідності дозволяє розбити складний шифр на набір простих підстановок, які легко піддаються зламу. Змодельована атака завершилася повним відновленням ключа та вихідного повідомлення, що підтверджує криптографічну незахищеність класичного шифру Віженера перед сучасними методами криптоаналізу.
Відповіді на питання:

1. Як змінювався індекс відповідності шифртексту при використанні ключів різної довжини? Поясніть отриманий результат. Зі збільшенням довжини ключа індекс відповідності шифртексту стрімко зменшувався від значення 0.05532 (для відкритого тексту) і наближався до значення рівноймовірного розподілу ($1/m \approx 0.03125$). Наприклад, для $r = 2$ він склав 0.04543, а для $r = 14$ впав до 0.03508. Це відбувається тому, що довгий ключ сильніше згладжує природні частоти мови, роблячи розподіл символів у шифртексті більш рівномірним.

2. За якими ознаками було визначено найбільш імовірну довжину невідомого ключа? Довжину ключа було визначено за двома головними ознаками:

- Середній індекс відповідності блоків (Avg_IC) для $r = 14$ склав 0.05528, що майже збігається з теоретичним показником відкритого тексту (0.05523).
- Статистика збігів символів (D_r) показала значний локальний максимум на $r = 14$ ($D_{14} = 396$), що суттєво перевищувало значення для некратних періодів (220–260).

3. Які альтернативні кандидати на довжину ключа ви досліджували, чому обрали саме їх та які результати отримали? Досліджувалися кандидати $r = 7, 14, 21, 28$, оскільки для цих значень (кратних 7) спостерігалися сплески індексу відповідності та кількості збігів. Значення $r = 7$ та $r = 21$ було відкинуто, оскільки їхній $Avg_IC \approx 0.044$ не досягав рівня відкритого тексту, що вказувало на те, що 7 є лише дільником справжньої довжини. Кандидат $r = 28$ також дав високий IC, але істинним періодом завжди є найменше базове значення, при якому IC сягає норми, тобто 14.

4. Яка статистична інформація використовувалася для відновлення окремих символів ключа? Використовувалися частоти появи символів у кожному з 14 блоків, на які був розбитий шифртекст. Найчастіші літери кожного блоку зіставлялися з найімовірнішими літерами російської мови («о», «е», «а»). Для точнішої оцінки застосовувався метод взаємного індексу відповідності між блоками.

5. Як можна перевірити правильність відновленого ключа, не знаючи початкового відкритого тексту? Правильність ключа підтверджується тим, що знайдений ключ є осмисленим словом чи фразою (у цьому випадку «последнийдозор»). Крім того, правильність перевіряється візуальним аналізом розшифрованого тексту: якщо він починає складатися зі змістовних слів (наприклад, «как я смог это сделать»), ключ підібрано вірно.

6. Як змінився результат розшифрування після внесення помилки в один символ ключа? Поясніть характер і періодичність змін. Після зміни першого символу ключа («п» на «р») результат розшифрування змінився строго періодично: неправильно розшифрувався рівно кожен 14-й символ (що відповідає довжині ключа). Було змінено 524 символи (7.15% тексту), при цьому зсув помилки був константним і становив 31 (або -1 mod32).

7. Які етапи проведеного криптоаналізу найбільше залежать від обсягу доступного шифртексту? Чому? Найбільше від обсягу тексту залежать етап визначення періоду r та етап частотного відновлення символів ключа. Якщо тексту мало, блоки стають занадто короткими. У коротких текстах природні частоти мови не встигають проявитися через статистичний шум, що робить індекс відповідності неточним і унеможливлює правильне зіставлення частот літер.

8. Які результати ваших експериментів найкраще демонструють слабкість шифру Віженера з періодичним ключем? Слабкість шифру найкраще демонструє експеримент із внесенням помилки в ключ. Той факт, що помилковий символ псує лише кожен r-й символ відкритого тексту (відсутність лавиноподібного ефекту), доводить, що шифр Віженера є лише набором незалежних шифрів Цезаря. Це дозволяє зламати його по стовпцях, аналізуючи кожну позицію ключа абсолютно незалежно від інших.